

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC KINH TẾ TP HỒ CHÍ MINH (UEH)
TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ



ĐỒ ÁN MÔN HỌC

ĐỀ TÀI

**ỨNG DỤNG THUẬT TOÁN DIJKSTRA TỐI ƯU
HÓA CHI PHÍ VẬN TẢI**

Học Phần: Cấu Trúc Dữ Liệu & Giải Thuật

Họ và tên	MSSV
LÊ MẬU PHỤC	31251025351
NGUYỄN PHAN TẤN KHOA	31251027185
HUỲNH ĐĂNG PHÁT	31231024121
NGUYỄN HOÀNG DỰ	31251026684

Giảng Viên: TS. Đặng Ngọc Hoàng Thành

Tp. Hồ Chí Minh, Ngày 05 tháng 05 năm 2026

MỤC LỤC

MỤC LỤC.....	2
CHƯƠNG 1. CÁC THUẬT TOÁN.....	4
1.1. Các Khái Niệm Liên Quan	4
a) Ứng dụng thực tế.....	4
b) Nguyên lý hoạt động cơ bản.....	4
c) Cơ sở của Thuật toán: Cấu trúc Đồ thị.....	5
1.2. Cấu Trúc và Cài Đặt Thuật Toán Dijkstra.....	8
a) Ý Tưởng Thuật Toán.....	8
b) Cấu Trúc Dữ Liệu Sử Dụng	8
c) Cài Đặt Thuật Toán.....	11
d) Độ Phức Tạp Thuật Toán	13
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ LỚP	14
2.1. Phân Tích Bài Toán	14
2.2. Sơ Đồ Lớp.....	15
2.3. Cài Đặt Lớp.....	15
a) Lớp Edge.....	15
b) Lớp Graph	16
CHƯƠNG 3. THIẾT KẾ GIAO DIỆN	17
3.1. Tổng quan giao diện.....	17
3.2. Mô Tả Các Thành Phần Giao Diện.....	17
a) ComboBox Chọn Điểm Bắt Đầu	17
b) ComboBox Chọn Điểm Đến	17
c) Ô Hiển Thị Lượng Nhiên Liệu Tiêu Thụ.....	18
d) Ô Hiển Thị Tổng Quãng Đường.....	18
e) Ô Hiển Thị Chi Phí Vận Chuyển	18
f) Ô Hiển Thị Thời Gian Di Chuyển	18
g) Nút Thực Hiện Tính Toán	18
h) Bảng Hiển Thị Chi Tiết Lộ Trình.....	19
i) Khu Vực Hiển Thị Bản Đồ	19
3.3. Đánh Giá Giao Diện.....	19
CHƯƠNG 4. THẢO LUẬN & ĐÁNH GIÁ	20
4.1. Các Kết Quả Nhận Được.....	20

4.2. Một Số Tồn Tại	20
4.3. Hướng Phát Triển	21
a) Tạo Bản Đồ Cụ Thể Địa Điểm	21
b) Đa Phương Tiện Vận Tải	21
c) Thêm Thông Tin Ảnh Hưởng Chi Phí Vận Tải Trên Bản Đồ	21
CHƯƠNG 5: PHỤ LỤC	22
5.1. Mã Nguồn GitHub	22
5.2. Hướng Dẫn Cài Đặt	22
5.3. Chi Tiết Mã Nguồn Giao Diện.....	30
TÀI LIỆU THAM KHẢO	44
BẢNG PHÂN CÔNG CÔNG VIỆC.....	45

CHƯƠNG 1. CÁC THUẬT TOÁN

1.1. Các Khái Niệm Liên Quan

Thuật toán Dijkstra là một kỹ thuật quan trọng trong khoa học máy tính, được phát triển bởi Edsger Dijkstra (nhà khoa học máy tính người Hà Lan). Nó giải quyết bài toán **tìm đường đi có chi phí/khoảng cách nhỏ nhất** từ một điểm xuất phát đến toàn bộ các điểm khác trong một mạng lưới.

a) Ứng dụng thực tế

- **Hệ thống GPS/Bản đồ số:** Google Maps, Apple Maps sử dụng thuật toán này để gợi ý tuyến đường nhanh nhất
- **Định tuyến mạng:** Các giao thức như OSPF (Open Shortest Path First) sử dụng nguyên lý tương tự để chuyển gói dữ liệu
- **Giao thông vận tải:** Lập kế hoạch logistik, tìm tuyến đường tiết kiệm chi phí cho các xe chở hàng
- **Trò chơi điện tử:** Tính toán đường đi cho nhân vật AI trong game

b) Nguyên lý hoạt động cơ bản

Thuật toán hoạt động theo **cơ chế lựa chọn tham lam (greedy)**: ở mỗi bước, nó chọn một điểm chưa được xử lý mà có **chi phí di chuyển từ điểm khởi đầu là nhỏ nhất**, sau đó cập nhật chi phí cho tất cả các điểm lân cận của nó. Quá trình này lặp lại cho đến khi tất cả các điểm được xử lý.

Ưu điểm:

- Đảm bảo tìm được đường đi tối ưu (với điều kiện mọi trọng số đều không âm)
- Dễ hiểu, dễ cài đặt
- Hiệu suất tốt với đồ thị nhỏ và vừa

Hạn chế:

- Không hoạt động với các cạnh có trọng số âm
- Chi phí tính toán cao với đồ thị rất lớn (có thể cải thiện bằng heap)

c) Cơ sở của Thuật toán: Cấu trúc Đồ thị

Định nghĩa và Thành phần

Đồ thị là một mô hình toán học dùng để **biểu diễn mối quan hệ giữa các thực thể**. Nó bao gồm hai thành phần chính:

1. Đỉnh (Vertex/Node)

Đỉnh đại diện cho các **đối tượng hoặc vị trí** trong hệ thống. Mỗi đỉnh thường được gán một **nhãn hoặc định danh duy nhất**.

Ví dụ minh họa:

- **Trong bản đồ giao thông:** Mỗi thành phố là một đỉnh
 - Đỉnh A: Hà Nội
 - Đỉnh B: Hải Phòng
 - Đỉnh C: Hải Dương
- **Trong mạng xã hội:** Mỗi người dùng là một đỉnh
 - Người A, Người B, Người C...
- **Trong mạng máy tính:** Mỗi máy chủ/máy tính là một đỉnh
 - Server 1, Server 2, Client 1...

2. Cạnh (Edge)

Cạnh là **đường nối giữa hai đỉnh**, biểu thị mối liên hệ hoặc khả năng di chuyển/tương tác giữa chúng. Cạnh thường mang một **trọng số (weight)** đại diện cho chi phí, khoảng cách, hoặc cường độ của mối liên kết.

Các loại cạnh theo hướng:

- **Cạnh vô hướng:** Mối liên hệ lưu thông hai chiều
 - Ví dụ: Đường xá giữa hai thành phố (có thể đi từ $A \rightarrow B$ hoặc $B \rightarrow A$ với cùng khoảng cách)
- **Cạnh có hướng:** Mối liên hệ một chiều
 - Ví dụ: Lượt follow trên mạng xã hội (A follow B không đồng nghĩa B follow A), hoặc các con đường một chiều

Ví dụ về trọng số cạnh:

- Hà Nội —(120 km)— Hải Phòng
- |
- L—(60 km)— Hải Dương
- Chi phí di chuyển từ Hà Nội → Hải Phòng = 120 km
- Chi phí di chuyển từ Hà Nội → Hải Dương = 60 km

Ví dụ Thực tế Chi Tiết

Bài toán: Tìm tuyến đường rẻ nhất

Giả sử bạn muốn đi từ **Thành phố A** đến **Thành phố F** qua mạng lưới đường xá:

- 50 km
- B ————— C
- / \ / \
- 30 km 20 km 15 km 35 km
- / \ / \
- A E ————— F
- \ / 40 km
- 60 km 25 km
- \ /
- D

Mô tả:

- A → B: 30 km
- A → D: 60 km
- B → C: 50 km
- B → E: 20 km
- D → E: 25 km
- C → E: 15 km
- C → F: 35 km
- E → F: 40 km

Câu hỏi: Tuyến đường ngắn nhất từ A đến F là bao nhiêu?

Áp dụng Dijkstra:

1. Bắt đầu từ A (khoảng cách = 0)
2. Xét B (30 km) và D (60 km), chọn B (nhỏ nhất)
3. Từ B, cập nhật: C = 80 km, E = 50 km
4. Chọn E (50 km) tiếp theo
5. Từ E, cập nhật: C = 65 km (tốt hơn 80), F = 90 km
6. Chọn C (65 km)
7. Từ C, cập nhật: F = 100 km (không tốt hơn 90)
8. Chọn F (90 km) → **Kết quả: A → B → E → F = 90 km**

Nhận xét quan trọng:

- Thuật toán Dijkstra luôn tìm được đường đi tối ưu nếu mọi trọng số không âm. Đây là lý do nó được tin tưởng trong các ứng dụng quan trọng
- Nếu đồ thị có cạnh với trọng số **âm**, thuật toán sẽ thất bại. Khi đó cần sử dụng Bellman-Ford
- Chi phí tính toán phụ thuộc vào kích thước đồ thị.
- Không chỉ dùng cho bản đồ, mà còn cho các bài toán tìm chi phí tối ưu trong mọi lĩnh vực (mạng, tài chính, logistics)
- Đồ thị cung cấp cách biểu diễn **trực quan và dễ hiểu** cho các mối quan hệ phức tạp, giúp con người dễ phân tích vấn đề

Kết luận: Thuật toán Dijkstra và cấu trúc đồ thị là những công cụ nền tảng trong khoa học máy tính, giúp giải quyết nhiều bài toán tối ưu hóa trong thế giới thực.

1.2. Cấu Trúc và Cài Đặt **Thuật Toán Dijkstra**

a) Ý Tưởng Thuật Toán

Thuật toán Dijkstra dựa trên **chiến lược lựa chọn tối ưu cục bộ** (greedy strategy): Ở từng vòng lặp, nó xác định đỉnh chưa được khám phá mà có chi phí tích lũy từ điểm khởi hành là nhỏ nhất, rồi sử dụng đỉnh này để điều chỉnh chi phí cho những đỉnh liền kề.

i) Quy Trình Chi Tiết

- **Giai Đoạn 1: Chuẩn Bị Ban Đầu**

Mục tiêu: Thiết lập trạng thái ban đầu cho quá trình tìm kiếm.

- Đặt **chi phí đến điểm khởi phát bằng 0** (vì ta đã ở đó)
- Khởi gán **chi phí đến mọi điểm còn lại thành vô hạn (∞)**, vì chưa tìm được con đường nào
- Xếp tất cả các đỉnh vào **danh sách "chưa được xử lý"** (unvisited set)

Ví dụ:

Điểm khởi phát: A

$$\text{dist}[A] = 0$$

$$\text{dist}[B] = \infty$$

$$\text{dist}[C] = \infty$$

$$\text{dist}[D] = \infty$$

Chưa xét: {A, B, C, D}

- **Giai Đoạn 2: Quá Trình Lặp Chính**

Mục tiêu: Liên tục cải thiện chi phí đến các điểm, từng bước giảm phạm vi điểm chưa khám phá.

Ở mỗi vòng lặp:

- Chọn điểm trụ cột: Từ tập hợp các đỉnh vẫn còn chưa được xử lý, tìm ra đỉnh u có giá trị chi phí nhỏ nhất
- Kiểm tra các hàng xóm: Với từng đỉnh v kề với u:
 - Tính toán con đường mới: chi phí qua u = $\text{dist}[u] + \text{trọng số}(u \rightarrow v)$
 - So sánh với con đường cũ: nếu con đường mới rẻ hơn $\text{dist}[v]$ hiện tại
 - Nếu tốt hơn: cập nhật $\text{dist}[v]$ bằng giá trị mới này

Minh họa:

Nếu: $\text{dist}[u] + w(u, v) < \text{dist}[v]$

Thì: $\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$

Ví dụ:

$\text{dist}[u] = 10, w(u \rightarrow v) = 5 \rightarrow$ chi phí qua u là 15

$\text{dist}[v] = 20$ (chi phí cũ)

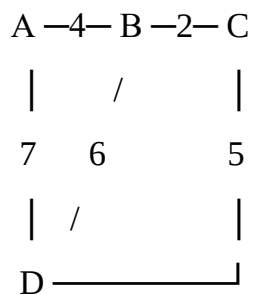
Vì $15 < 20 \rightarrow$ cập nhật $\text{dist}[v] = 15$

- **Giai Đoạn 3: Hoàn Tất**

Mục tiêu: Kết thúc quá trình khi đã xác định chi phí tối ưu đến toàn bộ điểm.

- Loại bỏ khỏi danh sách: Đánh dấu đỉnh u vừa xử lý là đã hoàn tất (không xét lại)
- Lặp lại quy trình: Quay trở lại bước 2, chọn điểm chưa xét tiếp theo
- Điều kiện dừng: Khi không còn điểm nào trong danh sách chưa xét

- **Ví dụ chi tiết**

Đồ thị mẫu:**Trọng số cạnh:**

- A-B: 4, A-D: 7
- B-C: 2, B-D: 6
- C-D: 5

Bước	Điểm Chọn	dist[A]	dist[B]	dist[C]	dist[D]	Chưa xét	Ghi chú
0	-	0	∞	∞	∞	{A,B,C,D}	Khởi tạo
1	A	0	4	∞	7	{B,C,D}	Cập nhật qua A

2	B	0	4	6	7	{C,D}	Cập nhật qua B: $C=4+2=6$
3	C	0	4	6	7	{D}	C không cải thiện D ($6+5=11>7$)
4	D	0	4	6	7	{}	Hoàn tất

Kết quả cuối cùng: Chi phí từ A đến tất cả các điểm

- A: 0
- B: 4 (đường $A \rightarrow B$)
- C: 6 (đường $A \rightarrow B \rightarrow C$)
- D: 7 (đường $A \rightarrow D$)

Các đặc điểm nổi bật

Đặc điểm	Giải thích
Lựa chọn tham lam	Mỗi bước chọn giải pháp tốt nhất lúc đó, nhưng điều này không làm tổn hại đến kết quả toàn cục
Độ chính xác	Với đồ thị có trọng số không âm, phương pháp luôn đạt được chi phí tối ưu
Hiệu ứng sóng lan	Chi phí được cải thiện dần dần từ điểm gần đến điểm xa, giống như sóng lan truyền
Thời gian thực thi	Phụ thuộc vào cấu trúc dữ liệu sử dụng: $O(n^2)$ với mảng, $O((n+m)\log n)$ với hàng đợi ưu tiên
Khả năng mở rộng	Có thể chạy song song hoặc với dữ liệu động, tuy cần điều chỉnh tùy theo ngữ cảnh cụ thể

- Thuật toán Dijkstra chứng tỏ rằng các lựa chọn tham lam có thể dẫn đến giải pháp tối ưu khi được thiết kế một cách khéo léo.

b) Cấu Trúc Dữ Liệu Sử Dụng

Để có thể triển khai thuật toán Dijkstra hiệu quả trong C#, các cấu trúc dữ liệu thường được sử dụng bao gồm:

- Mảng khoảng cách (dist[]): Lưu trữ khoảng cách ngắn nhất từ đỉnh nguồn đến từng đỉnh trong đồ thị.
- Mảng visited[]: Theo dõi đỉnh nào đã được xử lý hoàn toàn
- SortedSet hoặc PriorityQueue: Giúp lấy ra đỉnh có khoảng cách nhỏ nhất trong tập chưa xét một cách hiệu quả, giảm độ phức tạp từ $O(V^2)$ xuống $O((V + E) \log V)$.
- List<(int, int)>[] — Danh sách kề: Biểu diễn đồ thị, lưu trữ các đỉnh kề cùng trọng số tương ứng của mỗi cạnh.

c) Cài Đặt Thuật Toán

Cài đặt thuật toán Dijkstra bằng C# sử dụng PriorityQueue (có sẵn từ .NET 6):

```
using System;
using System.Collections.Generic;

class Dijkstra
{
    static int[] Dijkstra(List<(int dest, int weight)>[] graph, int start, int n)
    {
        int[] dist = new int[n];
        bool[] visited = new bool[n];
        for (int i = 0; i < n; i++) dist[i] = int.MaxValue;
```

```

dist[start] = 0;
var pq = new PriorityQueue<int, int>();
pq.Enqueue(start, 0);
while (pq.Count > 0)
{
    int u = pq.Dequeue();
    if (visited[u]) continue;
    visited[u] = true;
    foreach (var (v, weight) in graph[u])
    {
        if (!visited[v] && dist[u] != int.MaxValue
            && dist[u] + weight < dist[v])
        {
            dist[v] = dist[u] + weight;
            pq.Enqueue(v, dist[v]);
        }
    }
}
return dist;
}

static void Main()
{
    int n = 4; // Số đỉnh: 0=A, 1=B, 2=C, 3=D
    var graph = new List<(int, int)>[n];
    for (int i = 0; i < n; i++)
        graph[i] = new List<(int, int)>();
    graph[0].Add((1, 1)); // A -> B, w=1
    graph[0].Add((2, 4)); // A -> C, w=4

```

```

graph[1].Add((2, 2)); // B -> C, w=2
graph[1].Add((3, 5)); // B -> D, w=5
graph[2].Add((3, 1)); // C -> D, w=1
int[] result = Dijkstra(graph, 0, n);
string[] names = { "A", "B", "C", "D" };
Console.WriteLine("Khoảng cách ngắn nhất từ đỉnh A:");
for (int i = 0; i < n; i++)
    Console.WriteLine($" A -> {names[i]}: {result[i]}");
}
}

```

Kết quả đầu ra:

Khoảng cách ngắn nhất từ đỉnh A:

A -> A: 0

A -> B: 1

A -> C: 3

A -> D: 4

d) Độ Phức Tạp Thuật Toán

Cài đặt	Độ phức tạp thời gian	Ghi chú
Dùng mảng đơn giản	$O(V^2)$	Phù hợp đồ thị dày
Dùng PriorityQueue	$O((V + E) \log V)$	Phù hợp đồ thị thưa
Dùng Fibonacci Heap	$O(E + V \log V)$	Tối ưu nhất về lý thuyết

Trong đó V là số đỉnh và E là số cạnh của đồ thị.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ LỚP

2.1. Phân Tích Bài Toán

Đề tài: Ứng dụng thuật toán Dijkstra để tối ưu hoá chi phí vận tải

Trong lĩnh vực vận tải, việc lựa chọn lộ trình hợp lý có ảnh hưởng trực tiếp đến chi phí di chuyển. Mục tiêu bài toán đặt ra là xác định lộ trình có tổng chi phí vận tải nhỏ nhất trong số các lộ trình khả thi. Vì vậy, đề tài này tập trung ứng dụng thuật toán Dijkstra để xác định tuyến đường có chi phí vận tải thấp nhất giữa hai địa điểm trong hệ thống giao thông dựa trên các yếu tố chính:

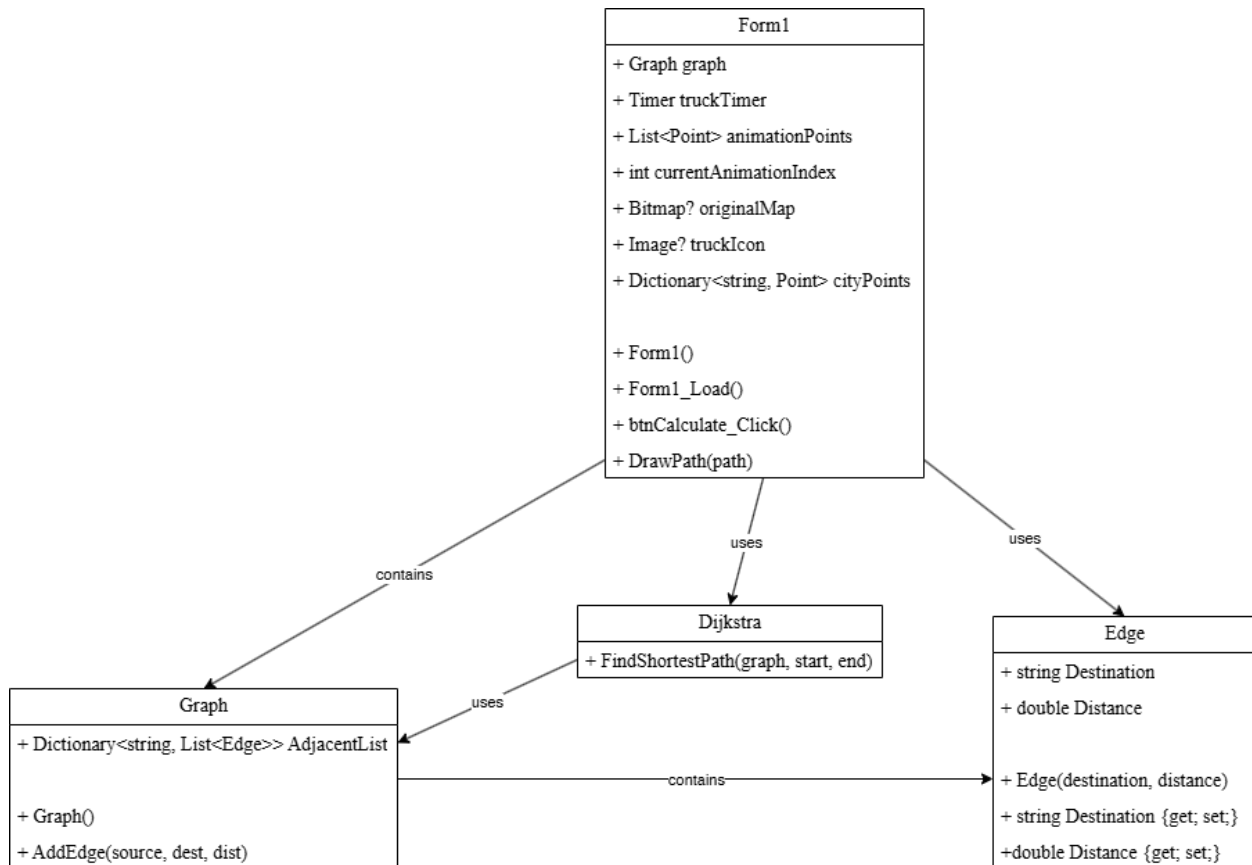
Hệ thống vận tải: Trong phần mềm bao gồm nhiều tỉnh thành khác nhau của Việt Nam như: Lai Châu, Điện Biên, Lào Cai, Sơn La, Hà Giang, Cao Bằng, Tuyên Quang, Thái Nguyên, Lạng Sơn, Quảng Ninh, Phú Thọ, Bắc Ninh, Hà Nội, Hải Phòng, Ninh Bình, Thanh Hóa, Nghệ An, Hà Tĩnh, Quảng Trị, Huế, Đà Nẵng, Quảng Ngãi, Gia Lai, Đắk Lak, Khánh Hòa, Lâm Đồng, Tây Ninh, Đồng Nai, TP Hồ Chí Minh, Bà Rịa Vũng Tàu, Đồng Tháp, Vĩnh Long, An Giang, Cần Thơ, Cà Mau.

Chi phí: Trong phạm vi đề tài, chi phí vận tải được xây dựng dựa trên các yếu tố sau:

1. Hệ số tiêu thụ: 0,15 lít/km
2. Hệ số chi phí: 25.000 VND/lít
3. Vận tốc trung bình: 50 km/h

$$\text{Chi phí} = \text{Quãng đường} \times 0,15 \times 25000$$

2.2. Sơ Đồ Lớp



2.3. Cài Đặt Lớp

a) Lớp Edge

Lớp Edge đại diện cho một cạnh nối giữa hai đỉnh trong cấu trúc dữ liệu đồ thị. Lưu trữ thông tin chi tiết như: tên đỉnh đích cạnh đó hướng tới và trọng số của cạnh đó.

Cài đặt:

```
public class Edge
{
    public string Destination { get; set; }

    public double Distance { get; set; }

    public Edge(string destination, double distance)
    {
        Destination = destination;
        Distance = distance;
    }
}
```

b) Lớp Graph

Lớp Graph đại diện cho toàn bộ cấu trúc đồ thị của ứng dụng. Lưu trữ thông tin chi tiết như: danh sách tất cả các đỉnh, danh sách các cạnh kết nối giữa các đỉnh và phương thức xây dựng đồ thị.

Cài đặt:

```
public class Graph
{
    public Dictionary<string, List<Edge>>
        AdjacencyList = new Dictionary<string, List<Edge>>();

    public void AddEdge(string source, string destination, double distance)
    {
        if (!AdjacencyList.ContainsKey(source))
        {
            AdjacencyList[source] = new List<Edge>();
        }

        if (!AdjacencyList.ContainsKey(destination))
        {
            AdjacencyList[destination] = new List<Edge>();
        }

        AdjacencyList[source].Add(new Edge(destination, distance));
        AdjacencyList[destination].Add(new Edge(source, distance));
    }
}
```


CHƯƠNG 3. THIẾT KẾ GIAO DIỆN

3.1. Tổng quan giao diện

Giao diện chính của chương trình được xây dựng bằng nền tảng Windows Forms với ngôn ngữ được sử dụng để lập trình là C#. Chương trình hỗ trợ người dùng nhấn để chọn vị trí điểm xuất phát và vị trí điểm đến để thực hiện tính toán và hiển thị trực quan tuyến đường vận chuyển tối ưu giữa các tỉnh/thành tại Việt Nam trên bản đồ. Ngoài ra, giao diện còn cung cấp khả năng hiển thị bản đồ trực quan, mô phỏng đường đi và thống kê các thông số liên quan đến chi phí vận tải.

Giao diện được chia thành ba khu vực chính:

- Khu vực hiển thị dữ liệu quãng đường.
- Khu vực tùy chọn và hiển thị kết quả tính toán.
- Khu vực hiển thị bản đồ và mô phỏng tuyến đường.

3.2. Mô Tả Các Thành Phần Giao Diện

a) *ComboBox Chọn Điểm Bắt Đầu*

comboStartPoint

- Chức năng: Cho phép người dùng lựa chọn tỉnh/thành làm điểm xuất phát cho hành trình vận chuyển.
- Thuộc tính chính:

```
this.comboStartPoint.FormattingEnabled = true;
```

```
this.comboStartPoint.Location = new System.Drawing.Point(275, 102);
```

```
this.comboStartPoint.Size = new System.Drawing.Size(187, 51);
```

b) *ComboBox Chọn Điểm Đến*

comboDestination

- Chức năng: Cho phép người dùng lựa chọn địa điểm đích cần vận chuyển tới.
- Thuộc tính chính:

```
this.comboDestination.FormattingEnabled = true;
```

```
this.comboDestination.Location = new System.Drawing.Point(275, 177);
```

```
this.comboDestination.Size = new System.Drawing.Size(186, 51);
```

c) Ô Hiển Thị Lượng Nhiên Liệu Tiêu Thụ

textBox1

- Chức năng: Hiển thị lượng nhiên liệu tiêu hao ước tính dựa trên tổng quãng đường di chuyển.
- Thuộc tính chính:

```
this.textBox1.Location = new System.Drawing.Point(275, 246);
```

```
this.textBox1.Size = new System.Drawing.Size(186, 45);
```

d) Ô Hiển Thị Tổng Quãng Đường

textBox2

- Chức năng: Hiển thị tổng chiều dài tuyến đường tối ưu mà thuật toán tìm được, đơn vị tính là kilomet (km).
- Thuộc tính chính:

```
this.textBox2.Location = new System.Drawing.Point(275, 306);
```

```
this.textBox2.Size = new System.Drawing.Size(186, 45);
```

e) Ô Hiển Thị Chi Phí Vận Chuyển

textBox4

- Chức năng: Hiển thị tổng chi phí vận chuyển được tính toán từ lượng nhiên liệu tiêu thụ.
- Thuộc tính chính:

```
this.textBox4.Location = new System.Drawing.Point(275, 370);
```

```
this.textBox4.Size = new System.Drawing.Size(186, 45);
```

f) Ô Hiển Thị Thời Gian Di Chuyển

textBox5

- Chức năng: Hiển thị thời gian dự kiến hoàn thành hành trình vận chuyển.
- Thuộc tính chính:

```
this.textBox5.Location = new System.Drawing.Point(275, 439);
```

```
this.textBox5.Size = new System.Drawing.Size(180, 45);
```

g) Nút Thực Hiện Tính Toán

btnCalculate

- Chức năng: Kích hoạt thuật toán Dijkstra để tìm tuyến đường ngắn nhất giữa hai địa điểm.
- Thuộc tính chính:

this.btnCalculate.Text = "Calculate";

this.btnCalculate.BackColor = System.Drawing.SystemColors.ActiveCaption;

h) Bảng Hiển Thị Chi Tiết Lộ Trình

dgvDistance

- Chức năng: Hiển thị chi tiết các chặng đường mà thuật toán tìm được, bao gồm điểm đi, điểm đến và khoảng cách giữa các điểm.
- Thuộc tính chính:

this.dgvDistance.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;

i) Khu Vực Hiển Thị Bản Đồ

picmap

- Chức năng: Hiển thị bản đồ Việt Nam và mô phỏng trực quan tuyến đường vận chuyển bằng hiệu ứng xe tải di chuyển.
- Thuộc tính chính:

this.picmap.SizeMode = PictureBoxSizeMode.StretchImage;

3.3. Đánh Giá Giao Diện

Giao diện chương trình được thiết kế theo hướng trực quan và dễ sử dụng, giúp người dùng thao tác thuận tiện trong quá trình thử nghiệm thuật toán tối ưu tuyến đường. Các thành phần chức năng được bố trí rõ ràng, hỗ trợ hiển thị đầy đủ thông tin cần thiết như quãng đường, chi phí, thời gian và mô phỏng trực tiếp trên bản đồ. Điều này giúp nâng cao khả năng quan sát và đánh giá hiệu quả của thuật toán trong bài toán tối ưu vận tải.

CHƯƠNG 4. THẢO LUẬN & ĐÁNH GIÁ

4.1. Các Kết Quả Nhận Được

Sau quá trình nghiên cứu, phân tích và tiến hành cài đặt, nhóm đã đạt được những kết quả đáng ghi nhận sau:

- **Về mặt lý thuyết:** Tìm hiểu sâu sắc và cài đặt thành công thuật toán Dijkstra vào cấu trúc dữ liệu đồ thị (Graph và Edge) để giải quyết bài toán đường đi ngắn nhất.
- **Về mặt xây dựng ứng dụng:** Phát triển thành công một phần mềm mô phỏng trên nền tảng C# (Windows Forms). Chương trình hoạt động ổn định, xử lý được đầu vào là điểm xuất phát và điểm đến do người dùng lựa chọn.
- **Về mặt giải quyết bài toán thực tế:** Ứng dụng không chỉ tìm ra tuyến đường tối ưu về khoảng cách mà còn tự động tính toán được các thông số quan trọng trong logistics bao gồm: tổng quãng đường (km), lượng nhiên liệu tiêu thụ ước tính, tổng chi phí vận chuyển và thời gian di chuyển dự kiến.
- **Về mặt giao diện (UI/UX):** Xây dựng được giao diện trực quan, thân thiện. Các thông số được tổ chức rõ ràng thông qua các Textbox. Đặc biệt, chương trình có khả năng trích xuất chi tiết từng chặng đường qua bảng DataGridView và mô phỏng trực quan tuyến đường di chuyển của xe tải ngay trên bản đồ Việt Nam (PictureBox).

4.2. Một Số Tồn Tại

Mặc dù chương trình đã đáp ứng được các yêu cầu cơ bản của đề tài, nhưng vẫn còn một số hạn chế nhất định khi đối chiếu với môi trường thực tế:

- **Dữ liệu tĩnh:** Đồ thị và trọng số (khoảng cách giữa các tỉnh/thành) hiện tại đang được thiết lập cứng (tĩnh) trong bộ nhớ, chưa tự động cập nhật được các tuyến đường mới xây dựng.
- **Chưa xét đến các yếu tố biến động:** Thuật toán Dijkstra hiện tại chỉ đang tối ưu hóa thuần túy dựa trên chiều dài quãng đường. Trong thực tế, chi phí vận tải còn phụ thuộc nặng nề vào tình trạng giao thông (kẹt xe), thời tiết, chất lượng mặt đường, hoặc các trạm thu phí BOT.
- **Hạn chế của bản đồ mô phỏng:** Việc hiển thị trên PictureBox chỉ mang tính chất mô phỏng đường chim bay hoặc tọa độ được định sẵn, chưa thể hiện được chi tiết độ cong lồi của từng cung đường thực tế.
- **Tính linh hoạt của phương tiện:** Cách tính toán nhiên liệu và chi phí đang dựa trên một mức trung bình cố định, chưa có sự phân loại và tùy chọn cho các tải trọng xe hay loại nguyên liệu khác nhau (xe tải nhẹ, xe container, xe đông lạnh...).

4.3. Hướng Phát Triển

Từ những hạn chế còn tồn tại, nhóm đề xuất một số hướng phát triển để phần mềm có thể hoàn thiện và ứng dụng sát hơn với thực tế ngành logistics:

a) Tạo Bản Đồ Cụ Thể Địa Điểm

- **Mở rộng quy mô đồ thị:** Thay vì chỉ dừng lại ở cấp độ các tỉnh/thành phố, hệ thống có thể được mở rộng số lượng đỉnh để đi sâu vào chi tiết các quận/huyện, cụm công nghiệp, hoặc các kho bãi cụ thể.
- **Tích hợp bản đồ số động (Dynamic Map):** Nâng cấp phần giao diện hiển thị bản đồ bằng cách nhúng trực tiếp các dịch vụ bản đồ số, cho phép người dùng phóng to/thu nhỏ (zoom in/out) và tương tác trực tiếp bằng cách click chọn tọa độ xuất phát/đích đến ngay trên giao diện bản đồ.

b) Đa Phương Tiện Vận Tải

- **Bổ sung tùy chọn phương tiện:** Thêm chức năng cho phép người dùng lựa chọn loại phương tiện vận chuyển (ví dụ: xe tải nhẹ, xe container, xe tải đông lạnh, v.v.).
- **Tính toán chi phí linh hoạt:** Mỗi loại phương tiện sẽ được hệ thống thiết lập một bộ thông số riêng biệt về vận tốc trung bình, định mức tiêu hao nhiên liệu/km và đơn giá bảo trì. Từ đó thuật toán sẽ linh hoạt trả về mức chi phí và thời gian tối ưu tương ứng với từng loại xe.

c) Thêm Thông Tin Ảnh Hưởng Chi Phí Vận Tải Trên Bản Đồ

- **Đa dạng hóa trọng số của đồ thị (Edge Weights):** Tích hợp thêm các yếu tố ngoại cảnh vào trọng số cạnh khi chạy thuật toán Dijkstra. Ví dụ: bổ sung chi phí qua các trạm thu phí (BOT), thời gian chậm trễ ước tính khi đi qua các điểm đen kẹt xe vào giờ cao điểm, hoặc các đoạn đường đang cấm tải.
- **Tùy chọn mục tiêu tối ưu:** Nâng cấp thuật toán để cung cấp cho người dùng nhiều chiến lược tìm đường: tìm "Tuyến đường ngắn nhất" (ưu tiên khoảng cách), "Tuyến đường tiết kiệm nhất" (tránh trạm thu phí dù xa hơn), hoặc "Tuyến đường nhanh nhất" (tránh kẹt xe). Điều này sẽ giúp ứng dụng hỗ trợ đắc lực hơn cho việc ra quyết định điều phối vận tải.

CHƯƠNG 5: PHỤ LỤC

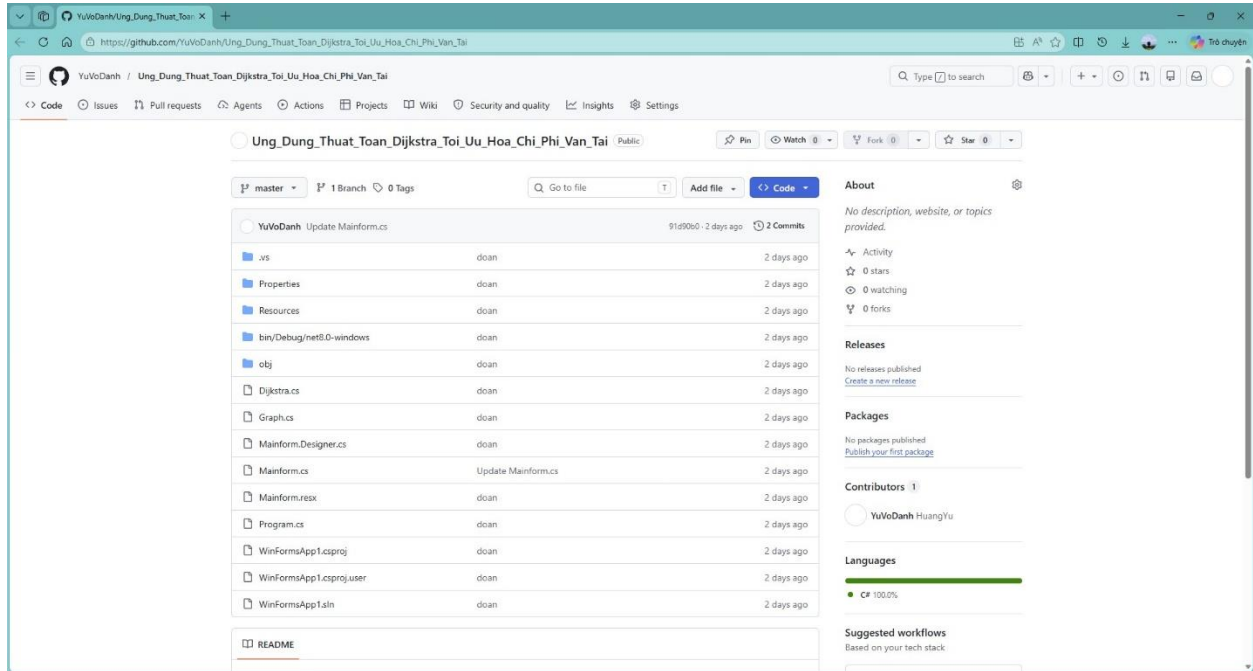
5.1. Mã Nguồn GitHub

Toàn bộ mã nguồn có thể tìm thấy tại:

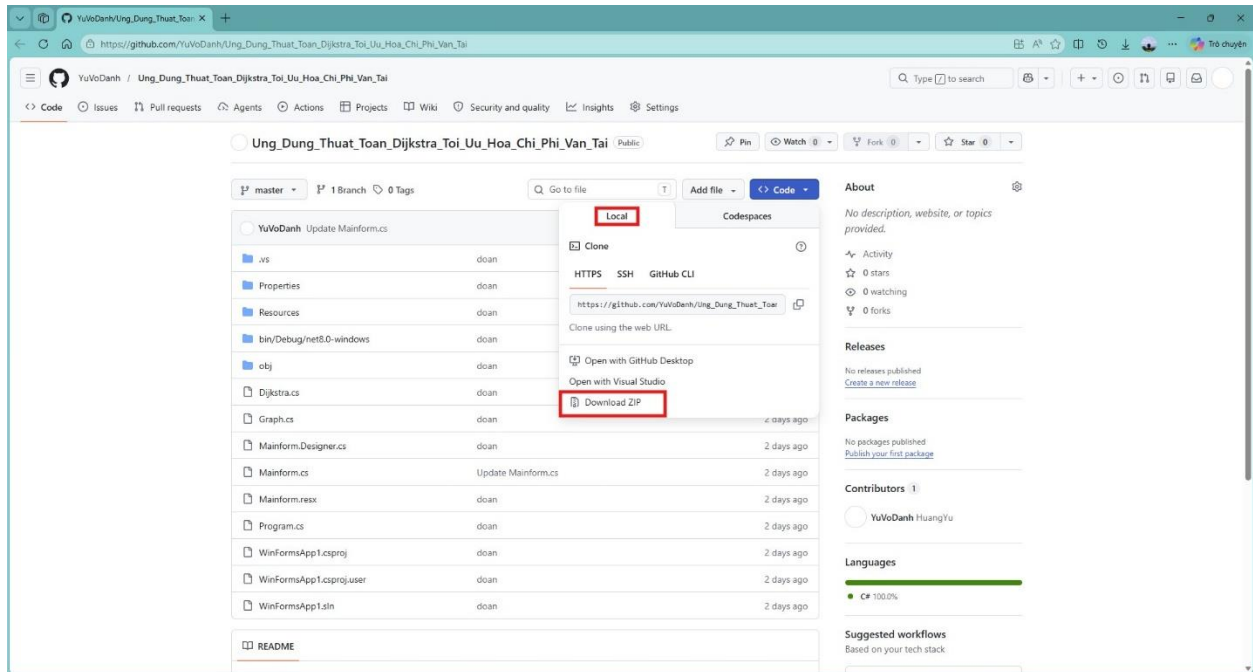
https://github.com/YuVoDanh/Ung_Dung_Thuat_Toan_Dijkstra_Toi_Uu_Hoa_Chi_Phi_Van_Tai.git

5.2. Hướng Dẫn Cài Đặt

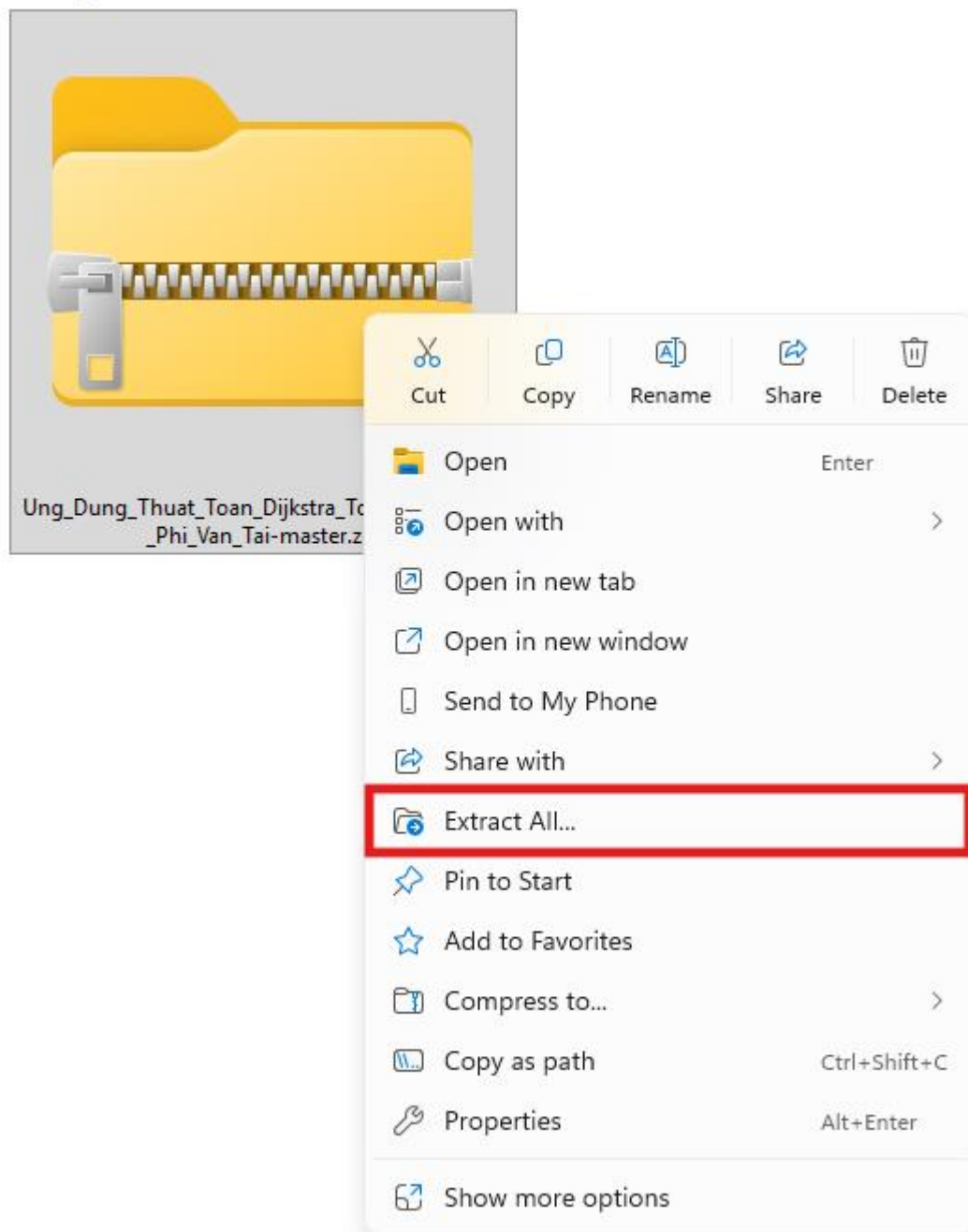
Bước 1: Truy cập đường link GitHub để xem dự án nhóm tải lên



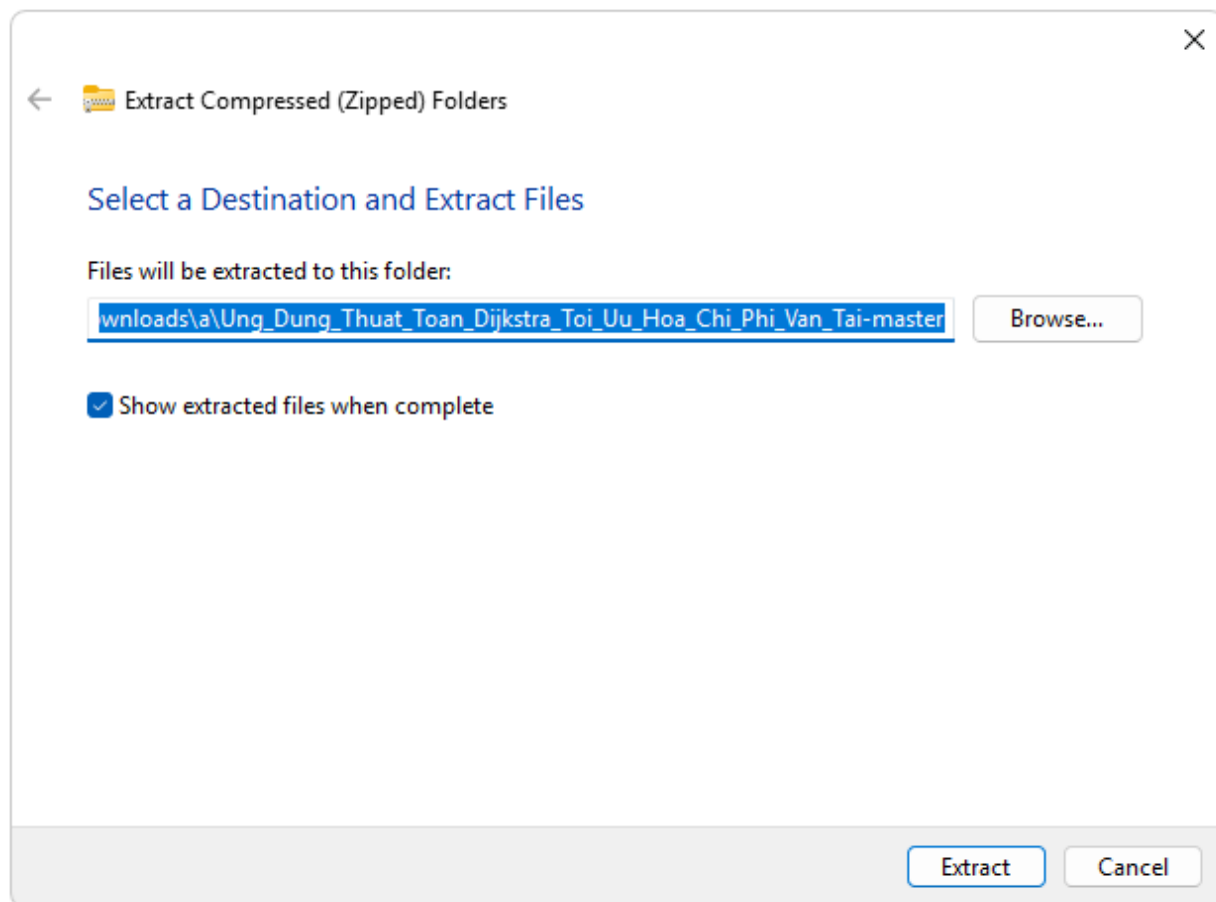
Bước 2: Bấm vào “Code”, chọn “Local” và chọn “Download ZIP” để tải về máy



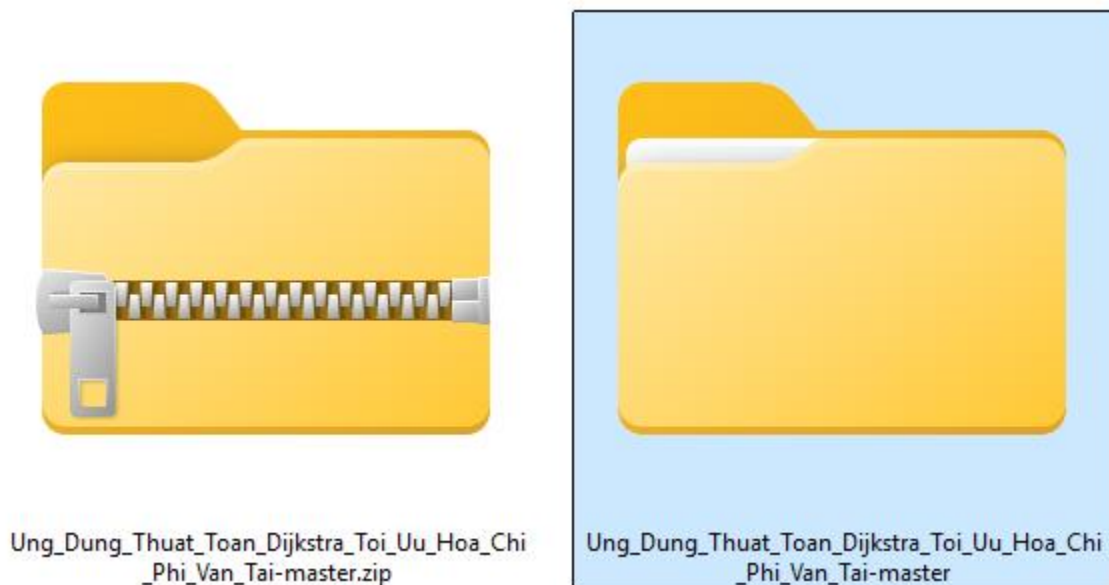
Bước 3: Tìm file vừa tải, nhấn chuột phải và chọn “Extract All”



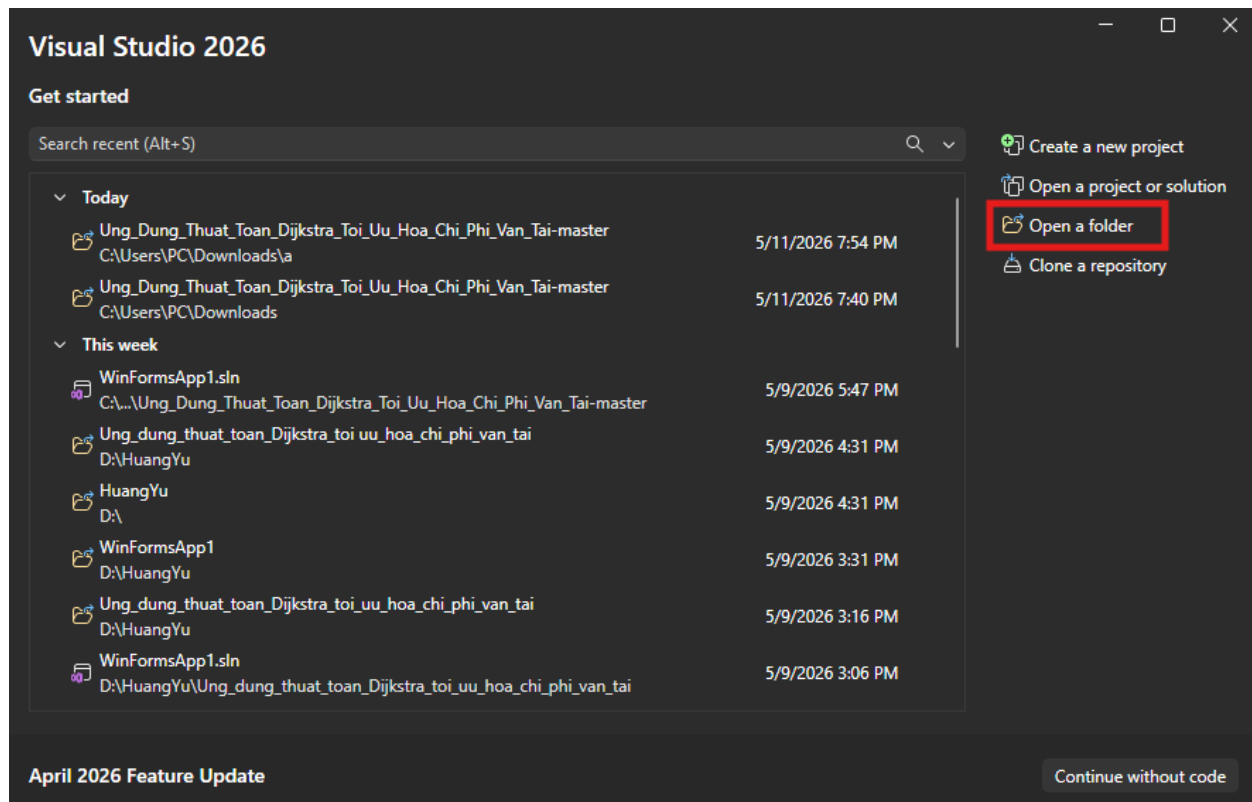
Bước 4: Kiểm tra địa điểm giải nén và bấm “Extract”



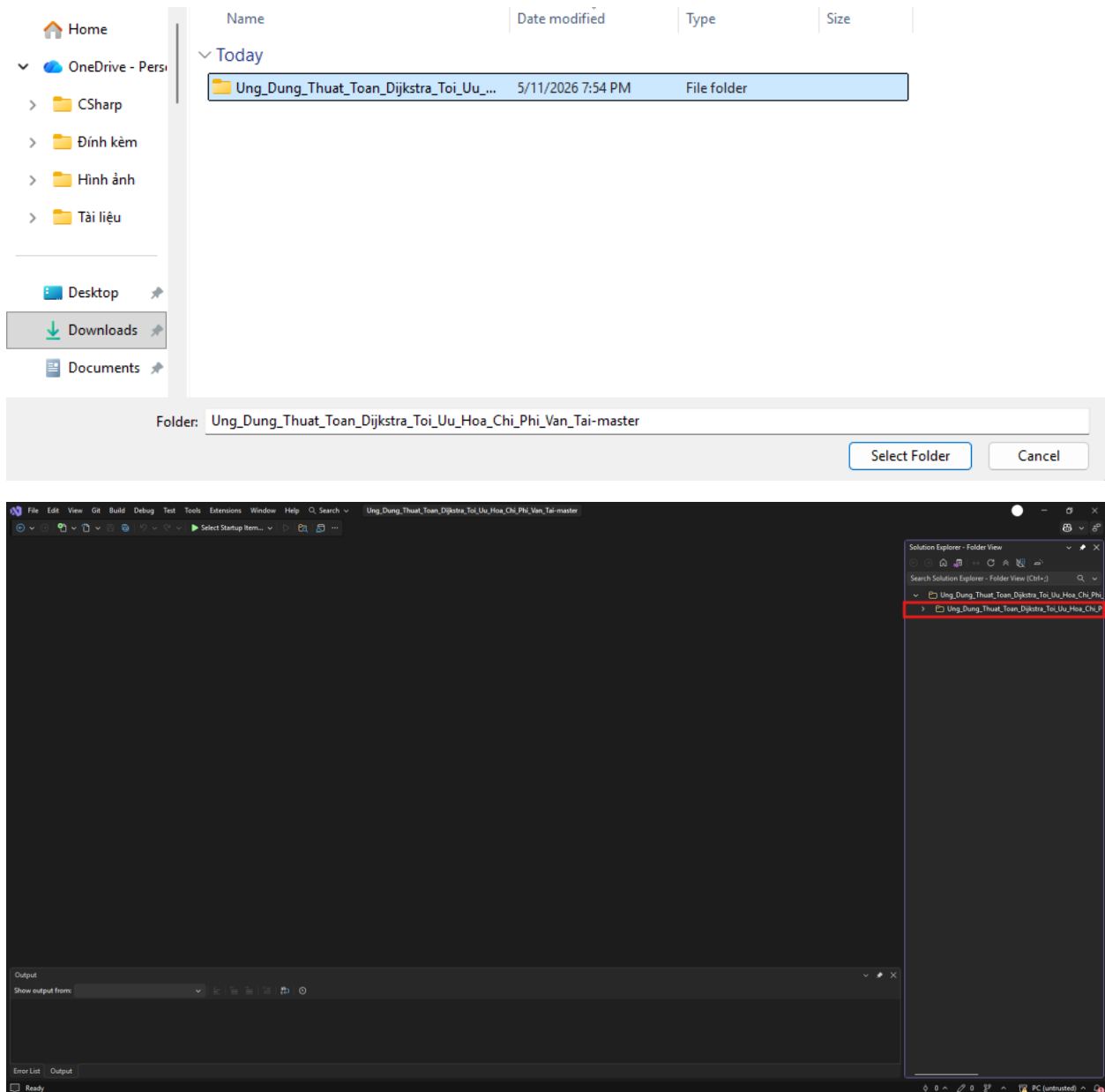
Sau khi giải nén sẽ được tệp tin như hình



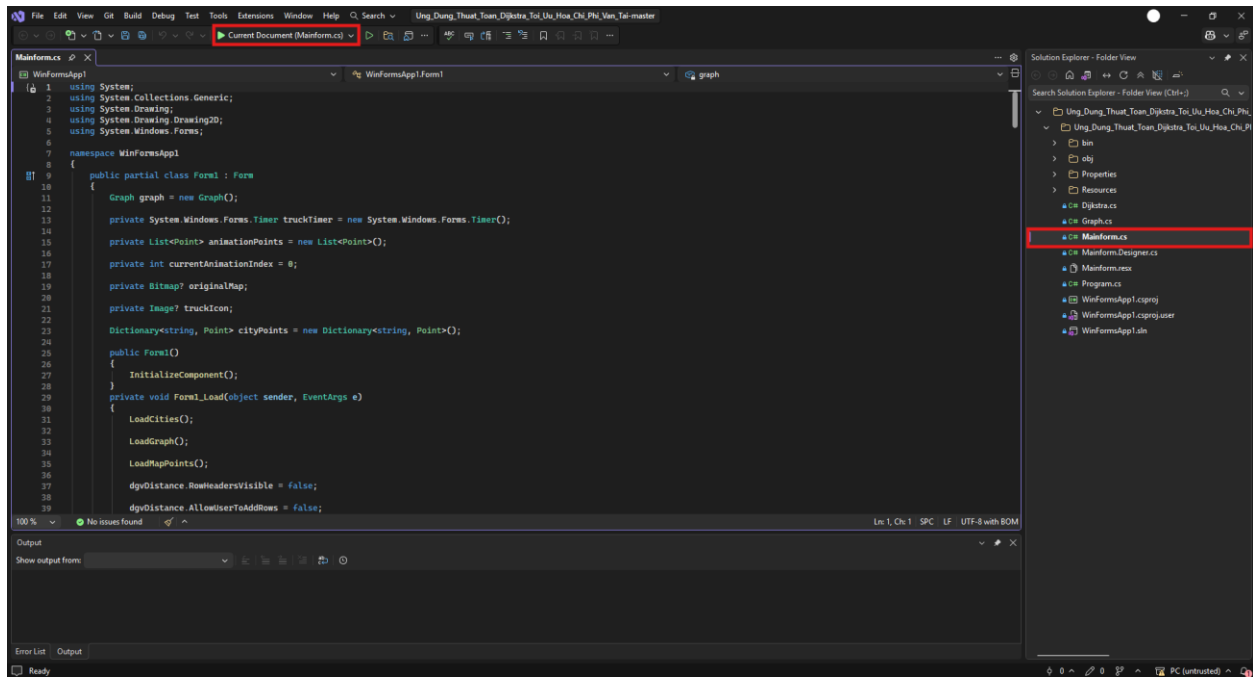
Bước 5: Mở “Visual Studio”, chọn “Open a folder”



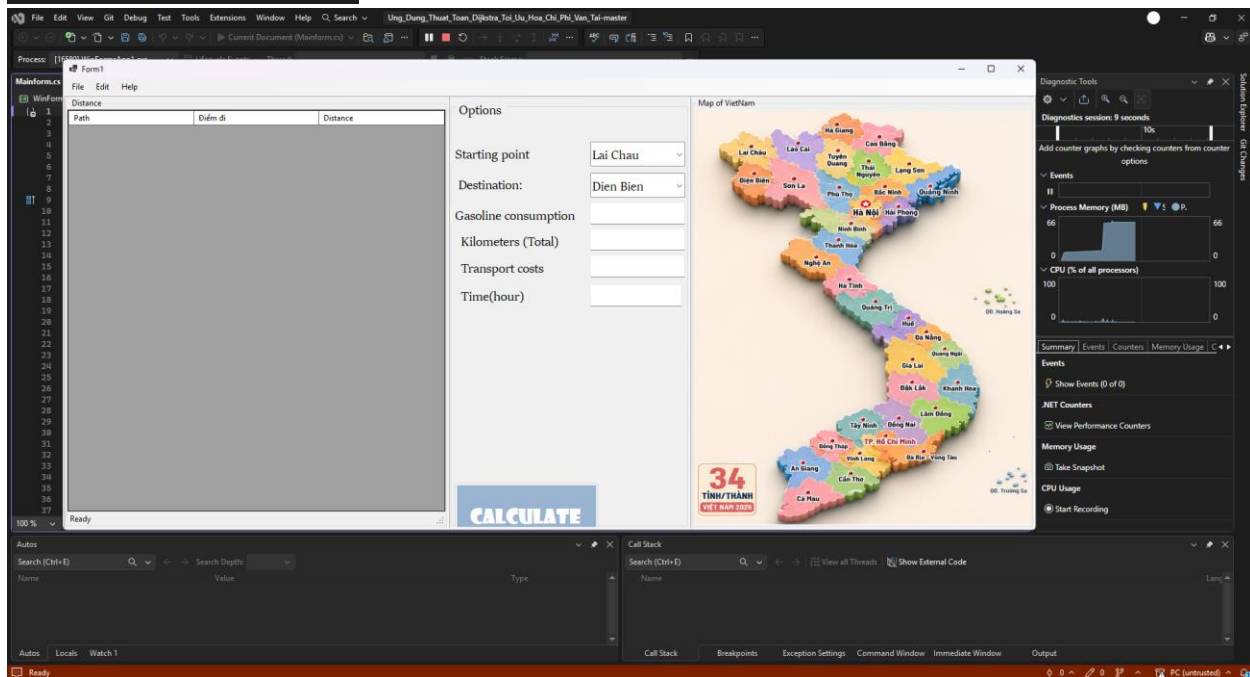
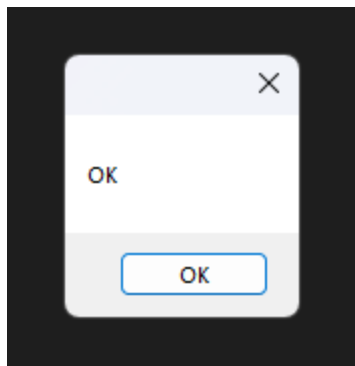
Bước 6: Chọn folder vừa giải nén và chọn “Select Folder”



Bước 7: Giao diện code hiện ra, nhấn đúp chọn “Mainform.cs”, nhấn “F5” trên bàn phím để chạy ứng dụng hoặc nhấn chọn “Current Document (Mainform.cs)” để chạy ứng dụng



Bước 8: Nhấn “OK” khi xuất hiện cửa sổ có nội dung “OK”



5.3. Chi Tiết Mã Nguồn Giao Diện

```
namespace WinFormsApp1
{
    partial class Form1
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed;
        otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            System.ComponentModel.ComponentResourceManager resources = new
            System.ComponentModel.ComponentResourceManager(typeof(Form1));

            menuStrip1 = new MenuStrip();
            fileToolStripMenuItem = new ToolStripMenuItem();
            saveToolStripMenuItem = new ToolStripMenuItem();
            openFileToolStripMenuItem = new ToolStripMenuItem();
            editToolStripMenuItem = new ToolStripMenuItem();
            clearToolStripMenuItem = new ToolStripMenuItem();
        }
    }
}
```

```
helpToolStripMenuItem = new ToolStripMenuItem();
aboutToolStripMenuItem = new ToolStripMenuItem();
toolStripMenuItem1 = new ToolStripMenuItem();
tableLayout = new TableLayoutPanel();
groupBoxDistance = new GroupBox();
dgvDistance = new DataGridView();
Column0 = new DataGridViewTextBoxColumn();
Column1 = new DataGridViewTextBoxColumn();
Column2 = new DataGridViewTextBoxColumn();
lblStatus = new StatusStrip();
groupBoxOptions = new GroupBox();
btnCalculate = new Button();
textBox5 = new TextBox();
textBox4 = new TextBox();
textBox2 = new TextBox();
textBox1 = new TextBox();
lblTime = new Label();
lblCosts = new Label();
lblKilometers = new Label();
lblGasoline = new Label();
label4 = new Label();
comboDestination = new ComboBox();
label2 = new Label();
comboStartPoint = new ComboBox();
groupBoxMap = new GroupBox();
picmap = new PictureBox();
backgroundWorker1 = new System.ComponentModel.BackgroundWorker();
menuStrip1.SuspendLayout();
tableLayout.SuspendLayout();
groupBoxDistance.SuspendLayout();
((System.ComponentModel.ISupportInitialize)dgvDistance).BeginInit();
groupBoxOptions.SuspendLayout();
```

```

groupBoxMap.SuspendLayout();
((System.ComponentModel.ISupportInitialize)picmap).BeginInit();
SuspendLayout();
//
// menuStrip1
//

menuStrip1.ImageScalingSize = new Size(22, 22);
menuStrip1.Items.AddRange(new ToolStripItem[] { fileToolStripMenuItem,
editToolStripMenuItem, helpToolStripMenuItem, toolStripMenuItem1 });
menuStrip1.Location = new Point(0, 0);
menuStrip1.Name = "menuStrip1";
menuStrip1.Size = new Size(1924, 31);
menuStrip1.TabIndex = 0;
menuStrip1.Text = "menuStrip1";
//
// fileToolStripMenuItem
//

fileToolStripMenuItem.DropDownItems.AddRange(new ToolStripItem[] {
saveToolStripMenuItem, openFileToolStripMenuItem });
fileToolStripMenuItem.Name = "fileToolStripMenuItem";
fileToolStripMenuItem.Size = new Size(51, 27);
fileToolStripMenuItem.Text = "File";
fileToolStripMenuItem.Click += fileToolStripMenuItem_Click;
//
// saveToolStripMenuItem
//

saveToolStripMenuItem.Name = "saveToolStripMenuItem";
saveToolStripMenuItem.Size = new Size(176, 30);
saveToolStripMenuItem.Text = "Save";
//
// openFileToolStripMenuItem
//

openFileToolStripMenuItem.Name = "openFileToolStripMenuItem";
openFileToolStripMenuItem.Size = new Size(176, 30);
openFileToolStripMenuItem.Text = "Open File";

```



```

//
// editToolStripMenuItem
//
editToolStripMenuItem.DropDownItems.AddRange(new ToolStripItem[] {
clearToolStripMenuItem });
editToolStripMenuItem.Name = "editToolStripMenuItem";
editToolStripMenuItem.Size = new Size(55, 27);
editToolStripMenuItem.Text = "Edit";
//
// clearToolStripMenuItem
//
clearToolStripMenuItem.Name = "clearToolStripMenuItem";
clearToolStripMenuItem.Size = new Size(143, 30);
clearToolStripMenuItem.Text = "Clear";
//
// helpToolStripMenuItem
//
helpToolStripMenuItem.DropDownItems.AddRange(new ToolStripItem[] {
aboutToolStripMenuItem });
helpToolStripMenuItem.Name = "helpToolStripMenuItem";
helpToolStripMenuItem.Size = new Size(61, 27);
helpToolStripMenuItem.Text = "Help";
//
// aboutToolStripMenuItem
//
aboutToolStripMenuItem.Name = "aboutToolStripMenuItem";
aboutToolStripMenuItem.Size = new Size(151, 30);
aboutToolStripMenuItem.Text = "About";
//
// toolStripMenuItem1
//
toolStripMenuItem1.Name = "toolStripMenuItem1";
toolStripMenuItem1.Size = new Size(31, 27);
toolStripMenuItem1.Text = " ";
//
// tableLayout

```

```

//
    tableLayout.CellBorderStyle =
TableLayoutPanelCellStyle.OutsetDouble;
    tableLayout.ColumnCount = 3;
    tableLayout.ColumnStyles.Add(new ColumnStyle(SizeType.Percent,
39.77725F));
    tableLayout.ColumnStyles.Add(new ColumnStyle(SizeType.Percent,
24.8308182F));
    tableLayout.ColumnStyles.Add(new ColumnStyle(SizeType.Percent,
35.4502869F));
    tableLayout.Controls.Add(groupBoxDistance, 0, 0);
    tableLayout.Controls.Add(groupBoxOptions, 1, 0);
    tableLayout.Controls.Add(groupBoxMap, 2, 0);
    tableLayout.Dock = DockStyle.Fill;
    tableLayout.Location = new Point(0, 31);
    tableLayout.Name = "tableLayout";
    tableLayout.RowCount = 1;
    tableLayout.RowStyles.Add(new RowStyle(SizeType.Percent, 100F));
    tableLayout.Size = new Size(1924, 1141);
    tableLayout.TabIndex = 2;
    tableLayout.Paint += tableLayoutPanel1_Paint;
//
// groupBoxDistance
//
    groupBoxDistance.Controls.Add(dgvDistance);
    groupBoxDistance.Controls.Add(lblStatus);
    groupBoxDistance.Dock = DockStyle.Fill;
    groupBoxDistance.Location = new Point(6, 6);
    groupBoxDistance.Name = "groupBoxDistance";
    groupBoxDistance.Size = new Size(754, 1129);
    groupBoxDistance.TabIndex = 0;
    groupBoxDistance.TabStop = false;
    groupBoxDistance.Text = "Distance";
//

```

```

// dgvDistance
//
dgvDistance.AutoSizeColumnsMode =
DataGridViewAutoSizeColumnsMode.Fill;
dgvDistance.ColumnHeadersHeightSizeMode =
DataGridViewColumnHeadersHeightSizeMode.AutoSize;
dgvDistance.Columns.AddRange(new DataGridViewColumn[] { Column0,
Column1, Column2 });
dgvDistance.Dock = DockStyle.Fill;
dgvDistance.Location = new Point(3, 26);
dgvDistance.Name = "dgvDistance";
dgvDistance.ReadOnly = true;
dgvDistance.RowHeadersWidth = 57;
dgvDistance.Size = new Size(748, 1078);
dgvDistance.TabIndex = 3;
dgvDistance.CellContentClick += dataGridView1_CellContentClick_1;
//
// Column0
//
Column0.HeaderText = "Path";
Column0.MinimumWidth = 7;
Column0.Name = "Column0";
Column0.ReadOnly = true;
//
// Column1
//
Column1.HeaderText = "Điểm đi";
Column1.MinimumWidth = 7;
Column1.Name = "Column1";
Column1.ReadOnly = true;
//
// Column2
//
Column2.HeaderText = "Distance";
Column2.MinimumWidth = 7;

```

```

Column2.Name = "Column2";
Column2.ReadOnly = true;
//
// lblStatus
//

lblStatus.ImageScalingSize = new Size(22, 22);
lblStatus.Location = new Point(3, 1104);
lblStatus.Name = "lblStatus";
lblStatus.Size = new Size(748, 22);
lblStatus.TabIndex = 2;
lblStatus.Text = "Ready";
lblStatus.ItemClicked += lblStatus_ItemClicked;
//
// groupBoxOptions
//

groupBoxOptions.Controls.Add(btnCalculate);
groupBoxOptions.Controls.Add(textBox5);
groupBoxOptions.Controls.Add(textBox4);
groupBoxOptions.Controls.Add(textBox2);
groupBoxOptions.Controls.Add(textBox1);
groupBoxOptions.Controls.Add(lblTime);
groupBoxOptions.Controls.Add(lblCosts);
groupBoxOptions.Controls.Add(lblKilometers);
groupBoxOptions.Controls.Add(lblGasoline);
groupBoxOptions.Controls.Add(label4);
groupBoxOptions.Controls.Add(comboDestination);
groupBoxOptions.Controls.Add(label2);
groupBoxOptions.Controls.Add(comboStartPoint);
groupBoxOptions.Dock = DockStyle.Fill;
groupBoxOptions.Font = new Font("Sitka Banner", 16.1194019F,
FontStyle.Regular, GraphicsUnit.Point, 0);
groupBoxOptions.Location = new Point(769, 6);
groupBoxOptions.Name = "groupBoxOptions";
groupBoxOptions.Size = new Size(468, 1129);

```

```

groupBoxOptions.TabIndex = 1;
groupBoxOptions.TabStop = false;
groupBoxOptions.Text = "Options";
groupBoxOptions.Enter += groupBoxOptions_Enter;
//
// btnCalculate
//

btnCalculate.BackColor = SystemColors.ActiveCaption;
btnCalculate.Font = new Font("Showcard Gothic", 22.02985F, FontStyle.Bold,
GraphicsUnit.Point, 0);
btnCalculate.ForeColor = SystemColors.ButtonHighlight;
btnCalculate.Location = new Point(6, 905);
btnCalculate.Name = "btnCalculate";
btnCalculate.Size = new Size(285, 156);
btnCalculate.TabIndex = 22;
btnCalculate.Text = "Calculate";
btnCalculate.UseVisualStyleBackColor = false;
btnCalculate.Click += btnCalculate_Click_1;
//
// textBox5
//

textBox5.Location = new Point(275, 439);
textBox5.Name = "textBox5";
textBox5.Size = new Size(180, 45);
textBox5.TabIndex = 19;
textBox5.TextChanged += textBox5_TextChanged;
//
// textBox4
//

textBox4.Location = new Point(275, 370);
textBox4.Name = "textBox4";
textBox4.Size = new Size(186, 45);
textBox4.TabIndex = 18;
textBox4.TextChanged += textBox4_TextChanged;

```

```

//
// textBox2
//
textBox2.Location = new Point(275, 306);
textBox2.Name = "textBox2";
textBox2.Size = new Size(186, 45);
textBox2.TabIndex = 16;
textBox2.TextChanged += textBox2_TextChanged;
//
// textBox1
//
textBox1.Location = new Point(275, 246);
textBox1.Name = "textBox1";
textBox1.Size = new Size(186, 45);
textBox1.TabIndex = 15;
textBox1.TextChanged += textBox1_TextChanged;
//
// lblTime
//
lblTime.AutoSize = true;
lblTime.Location = new Point(11, 439);
lblTime.Name = "lblTime";
lblTime.Size = new Size(152, 43);
lblTime.TabIndex = 14;
lblTime.Text = "Time(hour)";
lblTime.Click += label8_Click;
//
// lblCosts
//
lblCosts.AutoSize = true;
lblCosts.Location = new Point(11, 372);
lblCosts.Name = "lblCosts";
lblCosts.Size = new Size(195, 43);
lblCosts.TabIndex = 13;
lblCosts.Text = "Transport costs";
lblCosts.Click += label7_Click;
//
// lblKilometers
//
lblKilometers.AutoSize = true;

```

```

        lblKilometers.Location = new Point(11, 309);
        lblKilometers.Name = "lblKilometers";
        lblKilometers.Size = new Size(226, 43);
        lblKilometers.TabIndex = 12;
        lblKilometers.Text = "Kilometers (Total)";
        lblKilometers.Click += label6_Click;
        //
        // lblGasoline
        //
        lblGasoline.AutoSize = true;
        lblGasoline.Location = new Point(0, 249);
        lblGasoline.Name = "lblGasoline";
        lblGasoline.Size = new Size(273, 43);
        lblGasoline.TabIndex = 11;
        lblGasoline.Text = "Gasoline consumption\r\n";
        lblGasoline.Click += label5_Click_1;
        //
        // label4
        //
        label4.AutoSize = true;
        label4.Location = new Point(0, 105);
        label4.Name = "label4";
        label4.Size = new Size(178, 43);
        label4.TabIndex = 7;
        label4.Text = "Starting point";
        label4.Click += label4_Click;
        //
        // comboDestination
        //
        comboDestination.FormattingEnabled = true;
        comboDestination.Location = new Point(275, 177);
        comboDestination.Name = "comboDestination";
        comboDestination.Size = new Size(186, 51);
        comboDestination.TabIndex = 3;
        comboDestination.SelectedIndexChanged +=
comboBox2_SelectedIndexChanged;
        //
        // label2
        //
        label2.AutoSize = true;
        label2.Location = new Point(6, 177);
        label2.Name = "label2";
        label2.Size = new Size(157, 43);
        label2.TabIndex = 2;
        label2.Text = "Destination:";

```

```

label2.Click += label2_Click;
//
// comboStartPoint
//
comboStartPoint.FormattingEnabled = true;
comboStartPoint.Location = new Point(275, 102);
comboStartPoint.Name = "comboStartPoint";
comboStartPoint.Size = new Size(187, 51);
comboStartPoint.TabIndex = 1;
comboStartPoint.SelectedIndexChanged +=
comboStartPoint_SelectedIndexChanged;
//
// groupBoxMap
//
groupBoxMap.Controls.Add(picmap);
groupBoxMap.Dock = DockStyle.Fill;
groupBoxMap.Location = new Point(1246, 6);
groupBoxMap.Name = "groupBoxMap";
groupBoxMap.Size = new Size(672, 1129);
groupBoxMap.TabIndex = 2;
groupBoxMap.TabStop = false;
groupBoxMap.Text = "Map of VietNam";
groupBoxMap.Enter += groupBox3_Enter;
//
// picmap
//
picmap.Dock = DockStyle.Fill;
picmap.Image = (Image)resources.GetObject("picmap.Image");
picmap.Location = new Point(3, 26);
picmap.Name = "picmap";
picmap.Size = new Size(666, 1100);
picmap.SizeMode = PictureBoxSizeMode.StretchImage;
picmap.TabIndex = 2;
picmap.TabStop = false;
picmap.Click += picmap_Click;
//
// Form1
//
AutoScaleDimensions = new.SizeF(9F, 23F);
AutoScaleMode = AutoScaleMode.Font;
ClientSize = new Size(1924, 1172);
Controls.Add(tableLayout);
Controls.Add(menuStrip1);
MainMenuStrip = menuStrip1;
Name = "Form1";

```



```

        SizeGripStyle = SizeGripStyle.Show;
        Text = "Form1";
        Load += Form1_Load;
        menuStrip1.ResumeLayout(false);
        menuStrip1.PerformLayout();
        tableLayout.ResumeLayout(false);
        groupBoxDistance.ResumeLayout(false);
        groupBoxDistance.PerformLayout();
        ((System.ComponentModel.ISupportInitialize)dgvDistance).EndInit();
        groupBoxOptions.ResumeLayout(false);
        groupBoxOptions.PerformLayout();
        groupBoxMap.ResumeLayout(false);
        ((System.ComponentModel.ISupportInitialize)picmap).EndInit();
        ResumeLayout(false);
        PerformLayout();
    }

#endregion

private MenuStrip menuStrip1;
private ToolStripMenuItem fileToolStripMenuItem;
private ToolStripMenuItem saveToolStripMenuItem;
private ToolStripMenuItem openFileToolStripMenuItem;
private ToolStripMenuItem editToolStripMenuItem;
private ToolStripMenuItem clearToolStripMenuItem;
private ToolStripMenuItem helpToolStripMenuItem;
private ToolStripMenuItem aboutToolStripMenuItem;
private TableLayoutPanel tableLayout;
private GroupBox groupBoxDistance;
private System.ComponentModel.BackgroundWorker backgroundWorker1;
private GroupBox groupBoxOptions;
private ComboBox comboStartPoint;
private ComboBox comboDestination;
private Label label2;

private Label label4;
private Label lblTime;
private Label lblCosts;
private Label lblKilometers;
private Label lblGasoline;
private TextBox textBox2;
private TextBox textBox1;
private TextBox textBox5;
private TextBox textBox4;
private Button btnCalculate;

```

```

private GroupBox groupBoxMap;
private PictureBox picmap;
private StatusStrip lblStatus;
private ToolStripMenuItem toolStripMenuItem1;
private DataGridView dgvDistance;
private DataGridViewTextBoxColumn Column0;
private DataGridViewTextBoxColumn Column1;
private DataGridViewTextBoxColumn Column2;

private void tableLayoutPanel1_Paint(object sender, PaintEventArgs e)
{
}

private void dataGridView1_CellContentClick_1(object sender,
DataGridViewCellEventArgs e)
{
}

private void lblStatus_ItemClicked(object sender,
ToolStripItemClickedEventArgs e)
{
}

private void textBox6_TextChanged(object sender, EventArgs e)
{
}

private void label9_Click(object sender, EventArgs e)
{
}

private void textBox5_TextChanged(object sender, EventArgs e)
{
}

private void textBox4_TextChanged(object sender, EventArgs e)
{
}

private void textBox2_TextChanged(object sender, EventArgs e)
{
}

private void textBox1_TextChanged(object sender, EventArgs e)
{
}

```

```
}

private void label8_Click(object sender, EventArgs e)
{
}

private void label7_Click(object sender, EventArgs e)
{
}

private void label6_Click(object sender, EventArgs e)
{
}

private void label5_Click_1(object sender, EventArgs e)
{
}

private void label4_Click(object sender, EventArgs e)
{
}

private void comboBox2_SelectedIndexChanged(object sender, EventArgs e)
{
}

private void label2_Click(object sender, EventArgs e)
{
}

private void groupBox3_Enter(object sender, EventArgs e)
{
}
}
```

TÀI LIỆU THAM KHẢO

1. Andrew Troelsen & Philip Japikse, *Pro C# 10 with .NET 6: Foundational Principles and Practices in Programming*, Apress, 2022.
2. John Sharp, *Microsoft Visual C# Step by Step*, Microsoft Press, 2019.
3. Herbert Schildt, *C# 8.0: The Complete Reference*, McGraw-Hill Education, 2020.
4. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, MIT Press, 2009.
5. Robert Sedgewick & Kevin Wayne, *Algorithms*, Addison-Wesley Professional, 2011.
6. [Microsoft Learn – Windows Forms Documentation](#)
7. [Microsoft Learn – DataGridView Control in Windows Forms](#)
8. [Microsoft Learn – Graphics and Drawing in Windows Forms](#)
9. [GeeksforGeeks – Dijkstra's Shortest Path Algorithm](#)
10. [TutorialsPoint – Graph Data Structure and Algorithms](#)

BẢNG PHÂN CÔNG CÔNG VIỆC

THÀNH VIÊN	MSSV	NHIỆM VỤ	ĐÁNH GIÁ
LÊ MẬU PHỤC	31251025351	Thiết kế giao diện menu chính Phân tích, thiết kế lớp, cài đặt thuật toán và trình bày hướng phát triển Tọa độ các tỉnh thành	100%
NGUYỄN PHAN TẤN KHOA	31251027185	Khái niệm và trình bày thuật toán Thảo luận, đánh giá về các kết quả nhận được và trình bày các tồn tại và hạn chế Giải thích chi tiết các chức năng	100%
HUỖNH ĐĂNG PHÁT	31231024121	Khái niệm và trình bày thuật toán Thảo luận, đánh giá về các kết quả đạt được Đo khoảng cách	100%
NGUYỄN HOÀNG DỰ	31251026684	Thiết kế giao diện menu chính Phân tích, thiết kế lớp và cài đặt thuật toán Hướng dẫn cài đặt	100%

Đồ án Cấu trúc dữ liệu và giải thuật Nhóm 9

ORIGINALITY REPORT

26%

SIMILARITY INDEX

24%

INTERNET SOURCES

13%

PUBLICATIONS

%

STUDENT PAPERS

PRIMARY SOURCES

1

source.ijs.si

Internet Source

2%

2

www.coursehero.com

Internet Source

2%

3

csharp.hotexamples.com

Internet Source

1%

4

github.com

Internet Source

1%

5

www.slideshare.net

Internet Source

1%

6

pastebin.com

Internet Source

1%

7

www.experts-exchange.com

Internet Source

1%

8

Trần Việt Chương, Phan Tấn Quốc, Hà Hải Nam. "ĐỀ XUẤT MỘT SỐ THUẬT TOÁN HEURISTIC GIẢI BÀI TOÁN CÂY STEINER NHỎ NHẤT", FAIR - NGHIÊN CỨU CƠ BẢN VÀ ỨNG DỤNG CÔNG NGHỆ THÔNG TIN - 2017, 2017

Publication

1%

9

www.studocu.com

Internet Source

1%